

Herrera_Christian_Lab8

March 20, 2026

1 Lab 8 : Numpy and Random Variables (Questions)

2 Assignment Submission Guidelines

2.0.1 Steps to Submit Your Lab Assignments Using Google Colab

1. Submission Deadline:

- All assignments must be submitted **no later than 23:59 PM tonight**.
- Late submissions will not be accepted unless prior arrangements have been made by the TAs.

2. Complete Your Work:

- Finish your work in Google Colab and ensure that your notebook is saved.

3. Submit Your Assignment on Canvas:

- Log in to **Canvas** and navigate to the assignment submission page.

4. File Naming Convention:

- Please name your files as follows: Lastname_Firstname_AssignmentName
- Example: Alex_John_Lab8.ipynb and Alex_John_Lab8.pdf

5. Technical Issues:

- If you encounter any technical issues with Canvas or your submission, please contact the TAs immediately **before the deadline** to avoid penalties.

2.1 Questions

2.2 Question 1: (5 Points)

Write a NumPy code to create an array of integers from 1 to 5.

```
[75]: import numpy as np

arr = np.array([1, 2, 3, 4, 5])
print(arr)
```

```
[1 2 3 4 5]
```

2.3 Question 2: (5 Points)

Write a NumPy code to create a 2D array with shape (3, 4) containing all zeros.

```
[76]: multi_dim_arr = np.array([
        [0, 0, 0, 0],
        [0, 0, 0, 0],
        [0, 0, 0, 0]])

print(multi_dim_arr)
```

```
[[0 0 0 0]
 [0 0 0 0]
 [0 0 0 0]]
```

2.4 Question 3: (5 Points)

Write a NumPy code to create an array of 5 random floating-point numbers between 0 and 1.

```
[77]: random_arr = np.random.random(5)

print(random_arr)
```

```
[0.75411244 0.6041206 0.84558288 0.61936363 0.96869479]
```

2.5 Question 4: (5 Points)

Write a NumPy code to add two arrays `arr1` and `arr2` element-wise.

```
[78]: arr1 = np.array([1, 2, 3])
      arr2 = np.array([4, 5, 6])

      result_arr = arr1 + arr2
      print(result_arr)
```

```
[5 7 9]
```

2.6 Question 5: (5 Points)

Write a NumPy code to calculate the mean of all elements in an array `arr`.

```
[79]: mean_value = np.mean(random_arr) # Using previous array.  
      print(mean_value)
```

0.7583748690449467

2.7 Question 6: (5 Points)

Write a NumPy code to reshape a 1D array `arr` with shape (8,) into a 2D array with shape (2, 4).

```
[80]: arr = np.array([1, 2, 3, 4, 5, 6, 7, 8])  
  
      reshaped_arr = arr.reshape(2, 4)  
      print(reshaped_arr)
```

```
[[1 2 3 4]  
 [5 6 7 8]]
```

2.8 Question 7: (5 Points)

Write a NumPy code to calculate the dot product of two arrays `arr1` and `arr2`.

```
[81]: arr1 = np.array([1, 2, 3])  
      arr2 = np.array([4, 5, 6])  
  
      dot_product = np.dot(arr1, arr2)  
      print(dot_product)
```

32

2.9 Question 8: (5 Points)

Write a NumPy code to find the indices of the maximum and minimum values in an array `arr`.

```
[82]: max_index = np.argmax(arr)  
      min_index = np.argmin(arr)  
  
      print("Index of maximum value:", max_index)  
      print("Index of minimum value:", min_index)
```

```
Index of maximum value: 7  
Index of minimum value: 0
```

2.10 Question 9: (5 Points)

Write a NumPy code to calculate the element-wise square root of an array `arr`.

```
[83]: arr = np.array([1, 4, 9, 16, 25])  
  
      sqrt_arr = np.sqrt(arr)  
  
      print(sqrt_arr)
```

[1. 2. 3. 4. 5.]

2.11 Question 10: (5 Points)

Write a NumPy code to calculate the sum of each column in a 2D array `arr`.

```
[84]: arr = np.array([  
      [1, 2, 3],  
      [4, 5, 6],  
      [7, 8, 9]  
      ])  
  
      column_sums = arr.sum(axis=0)  
  
      print(column_sums)
```

[12 15 18]

2.12 Question 11: (5 Points)

Write a NumPy code to compute the element-wise multiplication of two arrays `arr1` and `arr2`.

```
[85]: arr1 = np.array([1, 2, 3])  
      arr2 = np.array([4, 5, 6])  
  
      result = arr1 * arr2  
  
      print(result)
```

[4 10 18]

2.13 Question 12: (5 Points)

Write a NumPy code to calculate the mean of each row in a 2D array `arr`.

```
[86]: arr = np.array([
        [1, 2, 3],
        [4, 5, 6],
        [7, 8, 9]
    ])

    row_means = np.mean(arr, axis=1)

    print(row_means)
```

[2. 5. 8.]

2.14 Question 13: (5 Points)

Write a NumPy code to sort the elements in an array `arr` in descending order.

```
[87]: arr = np.array([4, 2, 3, 5, 1])

    sorted_arr = np.sort(arr)[::-1]

    print(sorted_arr)
```

[5 4 3 2 1]

2.15 Question 14: (5 Points)

Write a NumPy code to calculate the matrix multiplication of two 2D arrays `arr1` and `arr2`.

```
[88]: arr1 = np.array([
        [1, 2],
        [3, 4]])

    arr2 = np.array([
        [5, 6],
        [7, 8]])

    result = np.matmul(arr1, arr2)

    print(result)
```

[[19 22]
 [43 50]]

2.16 Question 15: (5 Points)

Write a NumPy code to perform element-wise comparison between two arrays `arr1` and `arr2` and return a new array with True for elements that are greater in `arr1` and False otherwise.

```
[89]: arr1 = np.array([5, 2, 8, 1])
      arr2 = np.array([3, 4, 7, 1])

      result = arr1 > arr2

      print(result)
```

```
[ True False  True False]
```

2.17 Question 16: (5 Points)

Write a Python program using NumPy to solve the following system of linear equations:

```
[90]: 2x + 3y = 12
      4x - 2y = 6
```

```
Cell In[90], line 1
    2x + 3y = 12
    ^
SyntaxError: invalid syntax
```

```
[91]: A = np.array([
      [2, 3],
      [4, -2]])

      B = np.array([12, 6])

      solution = np.linalg.solve(A, B)

      print(solution)
```

```
[2.625 2.25 ]
```

2.18 Question 17: (5 Points)

Create an array of 20 random integers between 1 and 100. Use a NumPy conditional statement to replace all numbers greater than 50 with the number -1.

```
[92]: arr = np.random.randint(1, 101, 20)

# Replace every number greater than 50 with -1
arr[arr > 50] = -1

print(arr)
```

```
[31 -1 37 -1 -1 -1 -1 -1 14 -1 -1 20 -1  2  7  9 -1 -1 50 -1]
```

2.19 Question 18: (5 Points)

Create a 5x5 matrix with values ranging from 1 to 25. Extract a 3x3 sub-matrix from the center of this matrix.

```
[93]: matrix = np.array([
    [1, 2, 3, 4, 5],
    [6, 7, 8, 9, 10],
    [11, 12, 13, 14, 15],
    [16, 17, 18, 19, 20],
    [21, 22, 23, 24, 25]])

sub_matrix = matrix[1:4, 1:4]

print("Original 5x5 matrix:")
print(matrix)

print("Center 3x3 sub-matrix:")
print(sub_matrix)
```

Original 5x5 matrix:

```
[[ 1  2  3  4  5]
 [ 6  7  8  9 10]
 [11 12 13 14 15]
 [16 17 18 19 20]
 [21 22 23 24 25]]
```

Center 3x3 sub-matrix:

```
[[ 7  8  9]
 [12 13 14]
 [17 18 19]]
```

2.20 Question 19: (5 Points)

Create an array of 30 random integers between 10 and 100. Use NumPy to identify elements that are either multiples of 3 or multiples of 5 and replace them with -1.

[94]: *# Array of 30 random integers from 10 to 100.*

```
arr = np.random.randint(10, 101, 30)
```

```
print("Original array:")
```

```
print(arr)
```

```
# Check for multiples of 3.
```

```
multiples_of_3 = arr % 3 == 0
```

```
# Check for multiples of 5.
```

```
multiples_of_5 = arr % 5 == 0
```

```
# Combine both conditions.
```

```
condition = multiples_of_3 | multiples_of_5
```

```
# Replace those values with -1.
```

```
arr[condition] = -1
```

```
print("Updated array:")
```

```
print(arr)
```

Original array:

```
[ 73  37  73  96  28  80  43  31  31  92  84  27  61  40  40  60  70  44
  85  52  93  64  50  26 100  26  82  79  21  80]
```

Updated array:

```
[73 37 73 -1 28 -1 43 31 31 92 -1 -1 61 -1 -1 -1 -1 44 -1 52 -1 64 -1 26
 -1 26 82 79 -1 -1]
```

2.21 Question 20: (5 Points)

Complex Array Filtering: Create an array of 100 random floats. Use NumPy to find all values that are within one standard deviation from the mean of the array. Return these values sorted in ascending order.


```
[95]: # Array of 100 random flotas.
arr = np.random.rand(100)

# Find mean.
mean = np.mean(arr)

# STD of the array?
std_dev = np.std(arr)

# Lower and upper bounds for one standard deviation from the mean.
lower_bound = mean - std_dev
upper_bound = mean + std_dev

# Keep only the values that are within the range we want.
filtered_values = arr[(arr >= lower_bound) & (arr <= upper_bound)]

# Sort the values.
sorted_values = np.sort(filtered_values)

print("Original array:")
print(arr)

print("Mean:", mean)
print("Standard deviation:", std_dev)

print("Values within one standard deviation of the mean:")
print(sorted_values, len(sorted_values))
```

Original array:

```
[0.83883783 0.18912382 0.63899551 0.6449353 0.83976873 0.98721678
0.83485478 0.75578737 0.66944494 0.67972625 0.34333618 0.32442658
0.52846937 0.90862655 0.97745039 0.82164054 0.9119356 0.91309685
0.27866851 0.45280076 0.21888939 0.89739998 0.83932304 0.56811595
0.30463559 0.77439871 0.62114739 0.6823232 0.74862878 0.39747732
0.64502385 0.11204088 0.6643521 0.78848483 0.52966651 0.36381051
0.17856226 0.50901265 0.06901566 0.14185016 0.96502037 0.55014541
0.77649712 0.39733751 0.85033891 0.5218176 0.4351709 0.70309741
0.69405947 0.20729476 0.95589968 0.17615841 0.58665464 0.11317512
0.45100825 0.29933402 0.87562818 0.56915527 0.45521974 0.50025081
0.00149805 0.42868996 0.86190203 0.00413717 0.18604408 0.32082398
0.9603101 0.73103359 0.96767682 0.17800442 0.9001406 0.16316389
0.63854861 0.76372866 0.19787865 0.04824446 0.43180472 0.58191822
0.60176594 0.02883833 0.07543442 0.26964539 0.65409326 0.78188347
0.86665558 0.59691089 0.91392697 0.08364618 0.86443889 0.84040271
0.08428853 0.05011887 0.19012116 0.20172836 0.16669164 0.83857193
0.00357746 0.0563099 0.02686231 0.35111549]
```

Mean: 0.5158914069577935

Standard deviation: 0.3021657127818103

Values within one standard deviation of the mean:

```
[0.21888939 0.26964539 0.27866851 0.29933402 0.30463559 0.32082398
 0.32442658 0.34333618 0.35111549 0.36381051 0.39733751 0.39747732
 0.42868996 0.43180472 0.4351709  0.45100825 0.45280076 0.45521974
 0.50025081 0.50901265 0.5218176  0.52846937 0.52966651 0.55014541
 0.56811595 0.56915527 0.58191822 0.58665464 0.59691089 0.60176594
 0.62114739 0.63854861 0.63899551 0.6449353  0.64502385 0.65409326
 0.6643521  0.66944494 0.67972625 0.6823232  0.69405947 0.70309741
 0.73103359 0.74862878 0.75578737 0.76372866 0.77439871 0.77649712
 0.78188347 0.78848483] 50
```

[]: